



TECHNICAL DOCUMENTATION

Platform Technical Reference

Complete technical reference for the Cireta RWA Launchpad — architecture, API endpoints, smart contracts, database schema, deployment, and security model.

FastAPI · Python 3.11

Next.js 16 · React 19

ERC-3643 Tokens

Base L2 · Chain ID 8453

SQLAlchemy 2.0

Redis · arq Workers

NAVIGATION

Table of Contents

ARCHITECTURE & STACK

01	System Architecture	3
02	Technology Stack	4

API REFERENCE

03	API Reference — Auth & KYC	5
04	API Reference — Tokens & Sales	6
05	API Reference — Portfolio & Admin	7
06	API Reference — Compliance & Health	8

SMART CONTRACTS

07	Contract Architecture & Addresses	9
08	Contract Details & Modules	10

DATA & CONFIGURATION

09	Database Schema (Part 1)	11
10	Database Schema (Part 2)	12
11	Environment Variables	13

OPERATIONS

12	Deployment & Docker	14
13	Security Model	15
14	Background Workers	16
15	Testing Strategy	17
16	Troubleshooting Guide	18

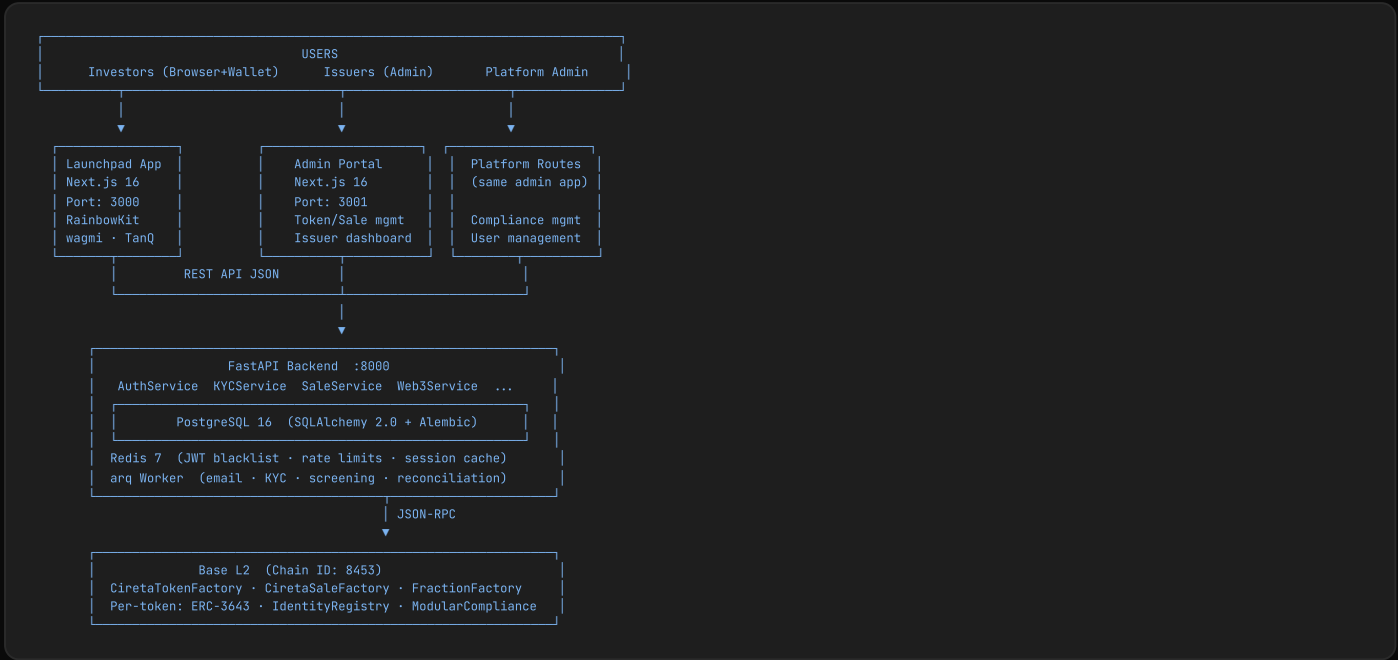
Admin Portal Screenshots

This document references the admin portal at admin.cireta.com — screenshots are included throughout to illustrate the management interface. The portal serves both issuer administrators and platform admins with role-scoped views.

CHAPTER 01

System Architecture

Cireta is a monorepo on Base L2. Request flow: Client → Middleware Stack → FastAPI Router → Service Layer → Repository → Database.



Monorepo Structure

```
cireta/
├─ apps/api/ FastAPI backend
├─ apps/launchpad/ Next.js investor
├─ apps/admin/ Next.js admin
├─ packages/common/ Shared Python lib
├─ contracts/ Hardhat ERC-3643
├─ infra/alembic/ DB migrations
├─ tests/ pytest suites
└─ docs/ Architecture docs
```

Design Principles

- Thin controllers — endpoints validate + delegate only
- Service layer holds ALL business logic
- Repository pattern — DB access abstracted
- Dependency injection via FastAPI Depends()
- EncryptedString/JSON for all PII at rest
- 300 LOC max per file / 150 per hook

CHAPTER 02

Technology Stack

Full stack breakdown across backend, frontend, blockchain, infrastructure, and tooling.

Backend

FastAPI (Python 3.11+)

SQLAlchemy 2.0

Alembic

Pydantic v2

asynccpg

uvicorn

Frontend

Next.js 16 (App Router)

React 19

TypeScript strict

Tailwind CSS 4

TanStack Query

Web3

RainbowKit

wagmi

viem

web3.py

Hardhat

OpenZeppelin

Identity

Sumsb KYC/AML

ONCHAINID

Chainlink PoR Feeds

Infra

PostgreSQL 16

Redis 7

arq (async worker)

Docker

Railway

Testing

pytest

Vitest

Playwright E2E

Hardhat tests

Storage

IPFS via Pinata

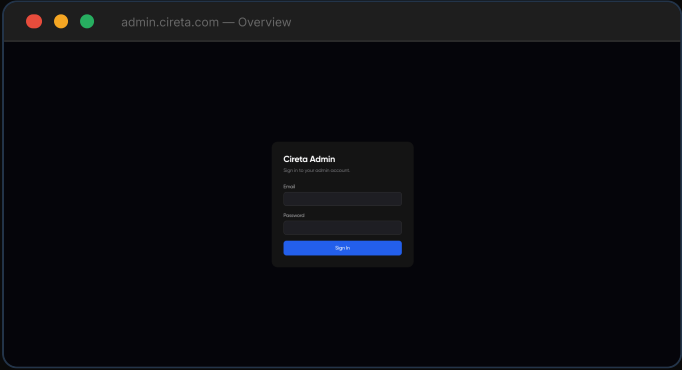
PostgreSQL encrypted fields

Redis session cache

Comms

Resend (transactional email)

arq job queue



Frontend Conventions

CSS variables only — no hardcoded colours. Atomic design: atoms → molecules → organisms → templates → pages. Atoms: forwardRef + extend HTML attrs. Never raw fetch() — all calls via lib/api/repositories/.

Mandatory Security Headers

X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin
Content-Security-Policy: default-src 'self'
HSTS: max-age=31536000 (prod only)

CHAPTER 03

API — Auth & KYC

Base URL: <https://api.cireta.com> · All responses: `application/json` · Errors: `{ "code": "ERROR_CODE", "message": "..." }`

Authentication

POST	/api/v1/auth/register	public · 10/min
POST	/api/v1/auth/login	public · 5/min
POST	/api/v1/auth/refresh	bearer token
POST	/api/v1/auth/logout	bearer token
GET	/api/v1/auth/me	bearer token

KYC

POST	/api/v1/kyc/initiate	bearer · kyc_level≥1
GET	/api/v1/kyc/status	bearer token
POST	/api/v1/kyc/webhook	HMAC-SHA256 verified · Sumsb only — NOT public

JWT Strategy

Token	TTL	Storage
Access JWT	15 min	Memory / Authorization header
Refresh Token	7 days	httpOnly cookie
Blacklist	Until expiry	Redis hash set

Token Rotation

On every refresh: old token invalidated, new one issued. Old hash stored in Redis for remaining TTL to prevent replay attacks.

Sumsb Webhook Validation

```
kyc/webhook.py — VALIDATE FIRST

import hmac, hashlib

def validate_sig(body: bytes, sig: str, secret: str) → bool:
    exp = hmac.new(
        secret.encode(), body,
        hashlib.sha256).hexdigest()
    return hmac.compare_digest(exp, sig)

body = await request.body()
sig = request.headers["X-App-Token"]
if not validate_sig(body,sig,SECRET):
    raise HTTPException(401, ...)
```

⚠ Critical Security Rule

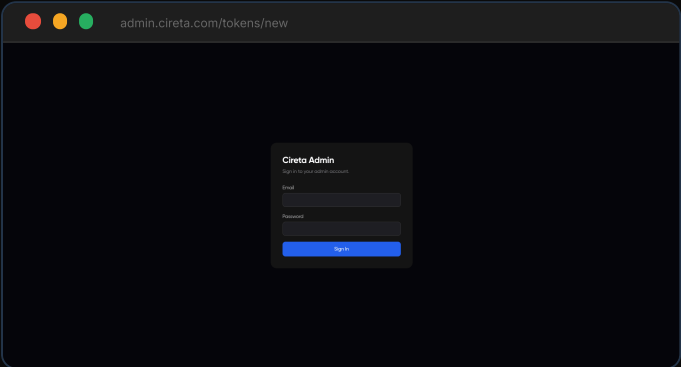
HMAC validation MUST run BEFORE any processing. Return 401 if invalid. Never process webhook payload before signature check.

CHAPTER 04

API — Tokens & Sales

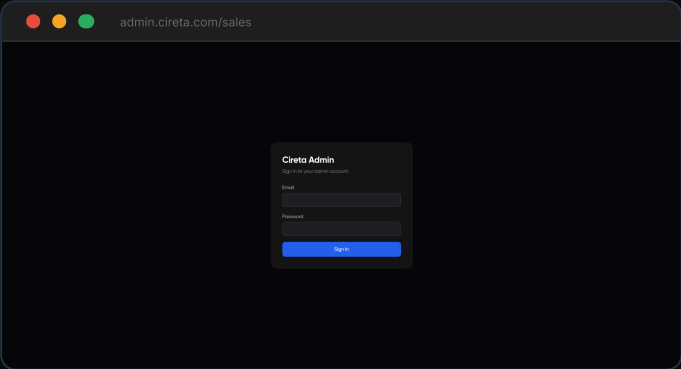
Token Endpoints

GET	/api/v1/tokens/	public
GET	/api/v1/tokens/{id}	public
POST	/api/v1/tokens/	issuer role
POST	/api/v1/tokens/{id}/deploy	issuer role



Sale Endpoints

GET	/api/v1/sales/	public
GET	/api/v1/sales/{id}	public
POST	/api/v1/sales/	issuer role
POST	/api/v1/sales/{id}/contribute	bearer · kyc≥2 · 20/min
POST	/api/v1/sales/{id}/finalize	issuer role
POST	/api/v1/sales/{id}/claim	bearer token
POST	/api/v1/sales/{id}/refund	bearer token



Rate Limits

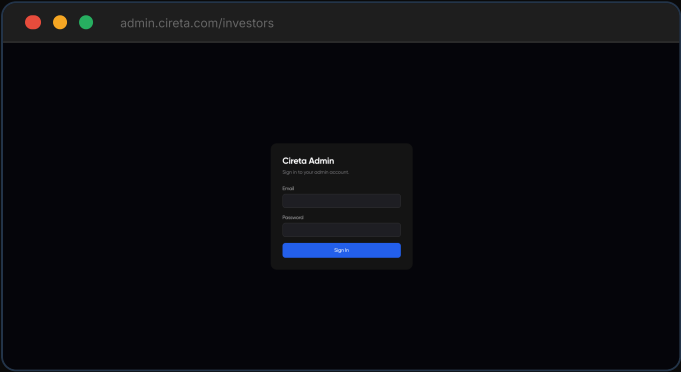
login: 5/min · register: 10/min · contribute: 20/min · default: 100/min. Redis sliding window. Exceeds return 429 with Retry-After header.

CHAPTER 05

API — Portfolio & Admin

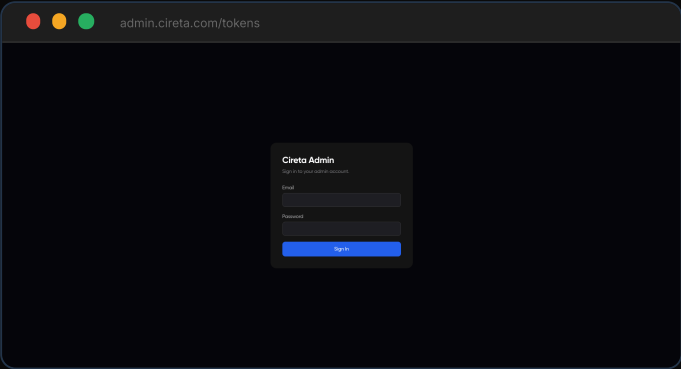
Portfolio Endpoints

GET	/api/v1/portfolio/holdings	bearer token
GET	/api/v1/portfolio/vesting	bearer token
POST	/api/v1/portfolio/vesting/{id}/claim	bearer
GET	/api/v1/portfolio/redemptions	bearer token
POST	/api/v1/portfolio/redemptions	bearer · kyc2



Admin Endpoints

GET	/api/v1/admin/issuers/	platform_admin
POST	/api/v1/admin/issuers/	platform_admin
PATCH	/api/v1/admin/issuers/{id}/fee	platform_admin
POST	/api/v1/admin/issuers/{id}/revoke	platform_admin



Role Hierarchy

investor — authenticated user with KYC · **issuer** — approved entity managing own tokens/sales · **platform_admin** — full access, manages issuers and compliance

CHAPTER 06

API — Compliance & Health

Compliance Endpoints

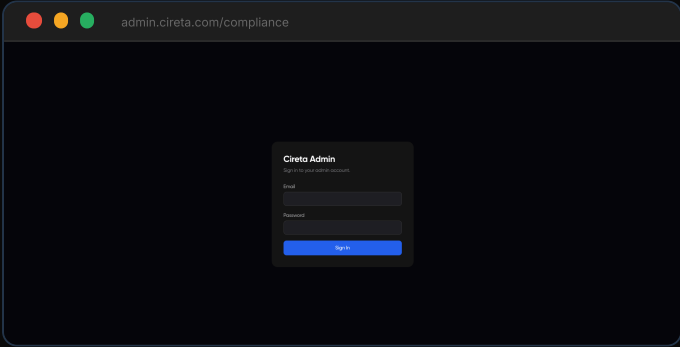
POST	/api/v1/admin/compliance/freeze	issuer admin
POST	/api/v1/admin/compliance/unfreeze	issuer admin
POST	/api/v1/admin/compliance/forced-transfer	issuer role
POST	/api/v1/admin/compliance/recover	issuer role
POST	/api/v1/admin/compliance/pause/{token_id}	issuer role
POST	/api/v1/admin/compliance/unpause/{token_id}	issuer role

Health Endpoints

GET	/api/v1/health/live	public — liveness probe
GET	/api/v1/health/ready	public — readiness probe (checks DB + Redis)

Audit Log Guarantee

Every compliance action writes an immutable row to `audit_logs`. Table is append-only — no UPDATE or DELETE ever issued. Fields: `actor_id`, `action`, `target_type`, `target_id`, `payload JSON`, `ip_address`, `created_at`.



Standard Error Format

```
{
  "code": "ERROR_CODE",
  "message": "Human readable text",
  "detail": {} // optional
}
```



```
// 400 - validation error
// 401 - not authenticated
// 403 - insufficient role/kyc
// 404 - resource not found
// 422 - Pydantic validation fail
// 429 - rate limit exceeded
// 500 - internal server error
```

Never expose stack traces

500 errors return generic message only. Full traceback written to structured logs. Sentry captures unhandled exceptions in production.

— CHAPTER 07

Smart Contract Architecture

Base Mainnet (Chain ID: 8453). Platform contracts deployed once; per-token contracts deployed via factory. All upgradeable via OpenZeppelin UUPS proxy.

Platform Contracts (Deployed Once)

CiretaTokenFactory

Deploys ERC-3643 compliant security tokens with IdentityRegistry + ModularCompliance

Issuer entry point for all token deployment. Each deployment creates: CiretaToken (ERC-3643), IdentityRegistry, ModularCompliance.

CiretaSaleFactory

0xe4a06Eaa949D12B173B0bA5f7CaABe473b4e8b5F

Deploys Sale contracts for each token sale. Manages USDC contributions, finalization, claim, and refund logic on-chain.

FractionFactory

0x224fa1965b5B8C1428eD5D92E6d04CF5967aE9ac

Deploys VaultImpl + FractionTokenImpl for vested sale mechanics. Manages vesting schedules and tranche releases on-chain.

PlatformFeeManager

0x545Ce9dc34E3086B505D9fd8DB443906E2c796f6

Collects and distributes platform fees from sales. Fee receiver: 0xBE84C7a8f44F673173d51C0A212C9C66267066A0. Default fee: 200 bps (2%).

Deployer Wallet

Contract deployment uses the key at ~/.ferron/x402-server-wallet.json. Must have ETH on Base Mainnet for gas. Keep this key in hardware storage — it controls all UUPS upgrade operations.

CHAPTER 08

Contract Details & Modules

Per-Token Contracts (via Factory)

Contract	Purpose
CiretaToken	ERC-3643 security token with transfer hooks
IdentityRegistry	KYC-approved wallet whitelist on-chain
ModularCompliance	Pluggable compliance rule enforcement
Sale	USDC contributions + finalize/claim/refund
VestingVault	Time-locked token release schedules
RedemptionManager	Token burn + USDC settlement

Compliance Modules

Module	Rule
CountryAllowModule	Restrict by investor country code
MaxOwnershipModule	Cap max % ownership per investor
MaxHolderCountModule	Limit total holder count
ConditionalTransferModule	Require approval for transfers
TimeTransfersLimitModule	Throttle transfer frequency

ONCHAINID Integration

Each KYC-approved investor gets an on-chain identity via ONCHAINID. Claims issued by Cireta's trusted issuer key, stored on identity contract.

```
// Claim Topics
CLAIM_KYC_APPROVED = 1
CLAIM_COUNTRY_CODE = 7
CLAIM_AML_CLEARED = 3

// Web3Service.issue_claim()
identity.addClaim(
  topic, scheme, issuer,
  signature, data, uri
)
```

Transfer Validation Flow

- Sender calls token.transfer(to, amt)
- Hook calls compliance.canTransfer(from,to,amt)
- Each compliance module checked in sequence
- Identity registry validates KYC claims for both addresses
- Transfer executes or reverts with reason

Upgradeability

All platform contracts use OpenZeppelin UUPS proxy. Upgrades require the deployer wallet with a 48-hour timelock enforced on-chain.

CHAPTER 09

Database Schema Part 1

PostgreSQL 16 · SQLAlchemy 2.0 mapped_column syntax · Alembic migrations · PII fields marked ENC

users

```
id UUID PK
email str unique indexed
hashed_pwd str
role enum(investor|issuer|admin)
kyc_status enum(none|pending|approved|rejected)
kyc_level int default 0
onchain_id str nullable
sumsub_id EncryptedString nullable
created_at datetime
updated_at datetime
```

issuers

```
id UUID PK
user_id UUID FK → users
name str
slug str unique
wallet_address EncryptedString
fee_bps int default 200
status enum(pending|active|suspended)
legal_entity_name str
jurisdiction str
created_at datetime
```

kyc_applications

```
id UUID PK
user_id UUID FK → users
sumsub_review EncryptedString
result_payload EncryptedJSON
status str
submitted_at datetime
reviewed_at datetime nullable
```

tokens

```
id UUID PK
issuer_id UUID FK → issuers
name / symbol str
asset_type enum(commodity|futures)
contract_address str nullable
chain_id int default 8453
total_supply Numeric
decimals int default 18
ipfs_docs_hash str nullable
chainlink_por str nullable
is_paused bool default false
created_at datetime
```

wallets

```
id UUID PK
user_id UUID FK → users
address EncryptedString
address_checksum str indexed + EIP-55
chain_id int (8453 = Base)
is_primary bool
created_at datetime
```

PII Encryption

EncryptedString / EncryptedJSON fields are AES-256 (Fernet) encrypted at ORM layer before write. Requires valid ENCRYPTION_KEY env var.

CHAPTER 10

Database Schema Part 2

token_sales

```
id UUID PK
token_id UUID FK → tokens
issuer_id UUID FK → issuers
payment_token str (USDC address)
soft_cap Numeric
hard_cap Numeric
status enum(draft|active|paused|
finalized|failed)
total_raised Numeric default 0
created_at datetime
```

sale_phases

```
id UUID PK
sale_id UUID FK → token_sales
phase_number int
name str
price_per_token Numeric
allocation Numeric
min_contribution Numeric
max_contribution Numeric
start_time datetime
end_time datetime
whitelist_only bool default false
created_at datetime
```

contributions

```
id UUID PK
user_id UUID FK → users
sale_id UUID FK → token_sales
phase_id UUID FK → sale_phases
amount Numeric
tokens_allocated Numeric
tx_hash str unique
status enum(pending|confirmed|
claimed|refunded)
claimed_at datetime nullable
created_at datetime
```

vesting_schedules

```
id UUID PK
token_id UUID FK → tokens
user_id UUID FK → users
total_amount Numeric
claimed_amount Numeric default 0
cliff_end datetime
vesting_end datetime
last_claim_at datetime nullable
created_at datetime
```

redemption_requests

```
id UUID PK
token_id UUID FK → tokens
user_id UUID FK → users
amount Numeric
fulfillment enum(physical|cash)
status enum(pending|processing|
fulfilled|cancelled)
tx_hash str
fulfilled_at datetime nullable
notes str nullable
created_at datetime
```

audit_logs — APPEND ONLY

```
id UUID PK
actor_id UUID FK → users
action str
target_type str
target_id str
payload JSON
ip_address str
created_at datetime
-- NEVER UPDATE OR DELETE ROWS
```

CHAPTER 11

Environment Variables

All config via the `Settings` class (Pydantic `BaseSettings`). Never hardcode secrets in source.

Core Application

ENVIRONMENT	development staging production
DATABASE_URL	postgresql+asyncpg://user:pass@host/db
REDIS_URL	redis://localhost:6379/0
CORS_ORIGINS	https://launchpad.cireta.com,https://admin.cireta.com

Security & Auth

JWT_SECRET_KEY	32+ char random (secrets.token_hex(32))
ENCRYPTION_KEY	Fernet key (Fernet.generate_key())

KYC / Sumsb

SUMSUB_SECRET_KEY	Sumsb dashboard → Apps → Secret
SUMSUB_APP_TOKEN	Sumsb dashboard → Apps → App token

Blockchain

WEB3_RPC_URL	https://mainnet.base.org (or Alchemy)
DEPLOYER_PRIVATE_KEY	~/.ferron/x402-server-wallet.json
TOKEN_FACTORY_ADDRESS	Deployed factory contract address
CHAIN_ID	8453 (Mainnet) 84532 (Sepolia)
PLATFORM_FEE_RECEIVER	0xBE84C7a8f44F673173d51C0A212C9C66267066A0

Third-Party Services

PINATA_API_KEY	IPFS pinning for legal documents
RESEND_API_KEY	Transactional email

Secret Generation

```
# JWT secret python -c "import secrets; \
print(secrets.token_hex(32))" # Fernet encryption key python -c
"from cryptography.fernet \ import Fernet; \
print(Fernet.generate_key().decode())"
```

⚠ Production Secrets

Never commit `.env` files. Use Railway secret injection. Rotating `ENCRYPTION_KEY` requires a full re-encryption migration — not a simple swap. Back up old key before rotating.

Env File Hierarchy

- `.env` — local dev (gitignored)
- `.env.test` — CI/test overrides
- Railway Dashboard → Environment Variables for staging and production
- Never commit `.env.production`

CHAPTER 12

Deployment & Docker

Multi-stage Docker builds. Deployed to Railway. All Dockerfiles at repo root. Non-root appuser in runner stage.

Dockerfile Pattern

```
## Dockerfile.api (multi-stage)
FROM python:3.11-slim AS deps
WORKDIR /app
COPY pyproject.toml poetry.lock ./
RUN pip install poetry && \
    poetry install --no-root --no-dev

FROM deps AS builder
COPY . .
RUN poetry install --no-dev

FROM python:3.11-slim AS runner
RUN useradd -m appuser
WORKDIR /app
COPY --from=builder /app .
USER appuser
EXPOSE 8000
CMD ["uvicorn", "apps.api.main:app", \
    "--host", "0.0.0.0", "--port", "8000"]
```

Audit Protocol (run after every step)

```
poetry run ruff check .
python -c "from apps.api.main import app"
poetry run pytest tests/ -x -q

cd apps/launchpad && npx tsc --noEmit
cd apps/admin && npx tsc --noEmit
cd contracts && npx hardhat test

docker build -f Dockerfile.api . \
    --no-cache
docker build -f Dockerfile.launchpad . \
    --no-cache
```

⚠ Migrations

Always run `alembic upgrade head` before starting the API in production. Never use `create_all()` — migrations only.

Services

Dockerfile	Service	Domain
Dockerfile.api	FastAPI	api.cireta.com
Dockerfile.launchpad	Investor app	launchpad.cireta.com
Dockerfile.admin	Admin app	admin.cireta.com

BUILD_LOG.md Protocol

After every build step, append to `docs/BUILD_LOG.md`: step name, files changed, audit results (pytest/ruff/tsc/docker), decisions/issues.

CHAPTER 13

Security Model

Security enforced at every layer: middleware headers, rate limits, data encryption, HMAC webhook validation, and on-chain compliance rules.

Middleware Stack (global)

Middleware	Responsibility
SecurityHeadersMiddleware	OWASP headers on every response
RateLimitMiddleware	Redis-backed sliding window
CORSMiddleware	Restrict to allowed origins
RequestLoggingMiddleware	Structured request/response logs
JWTAuthMiddleware	Token validation + blacklist check

Encrypted PII Fields

Table.Column	Type
users.sumsub_applicant_id	EncStr
wallets.address	EncStr
issuers.wallet_address	EncStr
kyc_applications.sumsub_review_id	EncStr
kyc_applications.result_payload	EncJSON

Rate Limits

Route Pattern	Limit
/auth/login	5/min per IP
/auth/register	10/min per IP
/sales/*/contribute	20/min per user
All other routes	100/min per IP

Wallet Address Rules

Checksum Enforcement (EIP-55)

Addresses stored checksummed in address_checksum for indexed lookups. Raw address stored encrypted. Always return checksummed on read.

HSTS Policy

Production Only

Strict-Transport-Security: max-age=31536000; includeSubDomains emitted only when ENVIRONMENT=production. Dev skips HSTS to allow HTTP.

CHAPTER 14

Background Workers

Two background processes alongside the API: arq job queue for async tasks and an event listener for on-chain sync (every ~12s).

arq Worker Jobs

Email Notifications

Triggered by: KYC status change, sale finalization, dividend declaration, vesting tranche ready, redemption status change. Uses Resend API with HTML templates.

KYC Processing

After Sumsb webhook: validate HMAC → parse result → update kyc_status + kyc_level → issue ONCHAINID claims → write audit_log → queue email.

AML Screening

Periodic re-screening of existing investors against updated sanctions lists. Triggers compliance freeze on new hits. Every run writes audit_log.

Reconciliation

Reconciles on-chain token balances with DB holdings. Flags discrepancies for review. Also reconciles pending contributions against confirmed tx hashes.

Event Listener (Chain Sync)

Block Polling — 12s interval

Listens for Base events every ~12s. Events tracked: Transfer, TokensClaimed, SaleFinalized, VestingClaimed, RedemptionFulfilled. Updates DB state accordingly.

Worker Config

```
## apps/api/worker.py
from arq import create_pool
from arq.connections import RedisSettings

class WorkerSettings:
    functions = [
        send_email_notification,
        process_kyc_result,
        run_aml_screening,
        reconcile_balances,
    ]
    redis_settings = RedisSettings(
        host=settings.redis_host
    )
    max_jobs = 10
    job_timeout = 300 # 5 minutes
```

Worker Deployment

Separate Railway service using same Docker image. CMD: arq apps.api.worker.WorkerSettings. Shares PostgreSQL + Redis with the API.

CHAPTER 15

Testing Strategy

Three-tier pyramid: unit tests for services, integration tests against real test DB, E2E via Playwright (85 tests).

Backend (pytest)

Unit Tests — tests/unit/

Every service method in isolation. External deps (Sumsb, Web3) mocked via pytest-mock. ComplianceMock for compliance logic. Target: 100% service coverage.

Integration Tests — tests/integration/

FastAPI TestClient + real PostgreSQL. Full request → service → DB round-trip. No DB mocks — real migrations applied. Sumsb/Web3 still mocked.

E2E Tests — tests/e2e/

Playwright. 85 tests covering full investor journey: register → KYC → wallet → contribute → claim → portfolio → redemption.

```
pytest tests/unit/ -q
pytest tests/integration/ -q
pytest tests/e2e/ -q --headed
```

Contract Tests (Hardhat)

Hardhat Test Suite

TypeScript tests. Deploy factory → token → run transfer/compliance scenarios → test vesting → test sale flow. Uses Hardhat Network (local Base fork).

```
cd contracts
npx hardhat test
npx hardhat coverage
```

Frontend (Vitest)

Component Tests

Vitest + React Testing Library for all atoms/molecules. Mock API repositories. Assert render, a11y, and event handling.

```
cd apps/launchpad && npx vitest run
cd apps/admin && npx vitest run
```

⚠️ CI Requirement

All 3 tiers must pass before merging to main. Results documented in BUILD_LOG.md. Failing test = blocked merge.

CHAPTER 16

Troubleshooting Guide

Common issues and resolutions. Start with: `railway logs --tail 100`

API Issues

Symptom	Cause & Fix
500 on startup	Missing env var — check Settings validation errors in logs
DB connection refused	Wrong DATABASE_URL — verify asyncpg:// format
KYC webhook 401	Bad HMAC — verify SUMSUB_SECRET_KEY matches dashboard
Token deploy fails	No ETH — top up deployer wallet on Base Mainnet
429 in tests	Redis not cleared — run redis-cli FLUSHDB between runs
JWT expired instantly	Clock skew — sync server time via NTP

Contract Issues

Symptom	Fix
Transfer reverts silently	Check IdentityRegistry — investor wallet may not be whitelisted
Claim fails post-finalize	Verify sale is finalized on-chain (not just DB)
UUPS upgrade fails	Caller must be contract owner — verify deployer wallet

Common Commands

```
## DB migrations
alembic current
alembic history --verbose
alembic upgrade head

## Redis inspection
redis-cli keys "*"
redis-cli ttl "jwt:blacklist:TOKEN"
redis-cli FLUSHDB # test env only!

## Reset test DB
dropdb cireta_test && createdb cireta_test
alembic upgrade head

## Contract state check
cd contracts
npx hardhat run scripts/check-status.ts \
  --network base

## Logs
railway logs --tail 50
railway logs --service api --tail 100

## Docker build validation
docker build -f Dockerfile.api . \
  --no-cache 2>&1 | tail -20
```

Structured Logs

All logs are structured JSON with: timestamp, level, service, request_id, user_id, endpoint, status_code, duration_ms. Filter with: `railway logs | jq 'select(.level=="ERROR")'`

Support

tech@cireta.com — infrastructure · **security@cireta.com** — incidents · GitHub Issues — tracked bugs



Cireta.

Real World Asset Tokenization Launchpad — Technical Reference v1.0

api.cireta.com

tech@cireta.com · security@cireta.com

This document is a confidential internal technical reference for authorised Cireta development team members and approved partners. Redistribution without explicit written permission from Cireta is prohibited. Contract addresses listed herein are for reference only — always verify against the official deployment registry before use. © 2026 Cireta. All rights reserved.